

INTEGRATION TEST

beforeAll 환경 구축 해설

issuer-service MockVASP 통합 테스트
7단계 인프라 조립 순서와 설계 의도

Hardhat

VASPServer

PostgreSQL · Redis · Java

COINCRAFT
ACADEMY

beforeAll — 7단계 인프라 조립

테스트 전체가 시작되기 전에 딱 한 번 실행 · 완료 후 모든 인프라가 연결된 상태



핵심 원칙: 내가 의존하는 것이 먼저 만들어져야 나를 만들 수 있다.

인프라(블록체인·DB·Redis) → 외부 서버(VASPServer) → 서비스 조립(Factory) 순서를 지킨다.

인프라 레이어

- Hardhat (블록체인)
- PostgreSQL (DB)
- Redis (Stream)
- Java (내부원장)

외부 서버 레이어

- VASPServer (HTTP)
- WebhookServer (HTTP)
- controlVasp (체인 제어)

서비스 레이어

- TokenIssuerFactory
- ChainEventListener
- ConsumerGroupPool

[1/7] Hardhat 노드 컨테이너

로컬 블록체인 · 포트 8545

```
hardhatContainer = await new GenericContainer('kyobo/hardhat-node:test')
    .withExposedPorts(8545)
    .withWaitStrategy(Wait.forLogMessage('Started HTTP and WebSocket JSON-RPC server'))
    .start();

await new JsonRpcProvider(hardhatRpc).send('hardhat_reset', []); // 체인 초기화
```

WaitStrategy가 핵심

컨테이너가 "뜬" 것과 RPC 서버가 "준비된" 것은 다르다.

로그 메시지가 찍힐 때까지 기다려야 다음 단계에서 연결이 성공한다.

hardhat_reset: 이전 실행 잔류 블록·TX 전부 제거.

매번 깨끗한 상태에서 시작하는 것을 보장한다.

이 단계가 완료되면

- `hardhatRpc` URL 확보
- 로컬 블록체인이 블록 생성 대기 상태
- Hardhat 기본 계정 20개 각 10,000 ETH 보유
- DEPLOYER_KEY / OPERATOR_KEY 즉시 사용 가능

[2/7] MockVASP 컨트랙트 배포

테스트 전용 NFT 발행 컨트랙트 · 세 가지 모드 지원

```
async function deployMockVASP(rpcUrl) {  
  const factory = new ContractFactory(abi, bytecode, deployer);  
  const contract = await factory.deploy(OPERATOR_ADDR);  
  await contract.waitForDeployment();  
  
  // deployer에게 OPERATOR_ROLE 부여 - setMode() 호출 전용  
  const OPERATOR_ROLE = ethers.keccak256(ethers.toUtf8Bytes('OPERATOR_ROLE'));  
  await mockVasp['grantRole'](OPERATOR_ROLE, DEPLOYER_ADDR);  
}
```

MockVASP 세 가지 모드

- **NORMAL** — issueActivityNFT 정상 실행
- **REVERT** — estimateGas 단계에서 revert
- **NO_EMIT** — 민팅 성공, Issued 이벤트 없음

왜 매번 새로 배포하나?

이전 실행의 NFT 잔액·nonce·상태가 남아 있으면 테스트가 오염된다.
매 실행마다 새 주소에 배포해서 완전히 초기화된 컨트랙트를 보장한다.

OPERATOR_ROLE 분리: 배포자(DEPLOYER_KEY)에게 모드 전환 권한을 부여한다. 실제 TX 서명은 OPERATOR_KEY가 담당 → nonce 충돌 없음.

[3/7] PostgreSQL + Java 컨테이너

Docker 내부 네트워크로 두 컨테이너 연결

```
dockerNetwork = await new Network().start(); // 컨테이너 간 내부 통신 채널

pgContainer = await new PostgreSQLContainer('postgres:16-alpine')
    .withNetwork(dockerNetwork)
    .withNetworkAliases('postgres') // Java가 이 호스트명으로 접근
    .start();

javaContainer = await new GenericContainer('kyobo/internal-ledger:test')
    .withEnvironment({ DB_URL: 'jdbc:postgresql://postgres:5432/kyobo_test' })
    .withWaitStrategy(Wait.forHttp('/api/internal/health', 8080).forStatusCode(200))
    .start();
```

Network alias 'postgres': Java 컨테이너는 PostgreSQL의 실제 IP를 모른다.
postgres 라는 이름으로만 접근한다. withNetworkAliases 가 그 이름을 등록한다.

HTTP 헬스체크 WaitStrategy: Spring Boot는 뜨는 데 수초가 걸린다.
/api/internal/health → 200 을 확인하고 나서야 다음 단계로 진행한다.

이 단계가 완료되면

PostgreSQL에 schema.sql 실행 완료 (issuance_requests, tx_mint_requests, mint_requests, processed_events, audit_log, reconcile_history ...)

Java internal-ledger 서버 기동 완료 → recordNftHolding / recordAuditLog 호출 가능

[4/7] Redis 컨테이너

Redis Stream 브로커 · 이벤트 흐름의 중간 버퍼

```
redisContainer = await new GenericContainer('redis:7-alpine')
    .withExposedPorts(6379)
    .withWaitStrategy(Wait.forLogMessage('Ready to accept connections'))
    .start();

redis = new Redis(redisUrl);
```

Redis가 하는 일

- WebhookServer → XADD → Stream 적재
- Consumer Group XREADGROUP 소비
- IdempotencyStore (중복 처리 방지)
- DLQ 스트림 보관

두 Consumer Group

- **activity-consumers** — 교보 앱 이벤트 → IssuerService
- **nft-consumers** — VASP 콜백 → PgNFTLedgerService
- 같은 스트림 키, 독립적으로 소비

Redis가 없으면 WebhookServer가 이벤트를 받아도 Consumer에게 전달할 수단이 없다. 인프라 레이어에서 가장 늦게 뜨지만 서비스 조립 전에 반드시 준비돼야 한다.

[5/7] VASPServer + controlVasp

두 지갑 키를 분리하는 이유 — nonce 충돌 방지

```
// OPERATOR_KEY - TX 서명·브로드캐스트 전용
vaspServer = new VASPServer({ signerKey: OPERATOR_KEY, callbackUrl: `http://localhost:${WEBHOOK_PORT}`, ... });
await vaspServer.start();

// DEPLOYER_KEY - setMode / freezeMining / snapshot 전용 (nonce 분리)
controlVasp = new AnvilVASPAdapter({ privateKey: DEPLOYER_KEY, mockVaspAddr, ... });
```

왜 키를 분리하는가?

같은 키를 쓰면 두 곳이 동시에 TX를 전송할 때 nonce가 충돌한다.

VASPServer는 `NonceManager` 로 nonce를 연속 관리.

controlVasp는 단발성 TX만 전송 (setMode 등).

키를 나누면 두 nonce 시퀀스가 완전히 독립된다.

	VASPServer	controlVasp
키	OPERATOR_KEY	DEPLOYER_KEY
역할	TX 서명·브로드캐스트	모드 전환·체인 제어
nonce	NonceManager 연속 관리	단발성 TX
메서드	issueActivityNFT()	setMode / freezeMining / snapshot

beforeEach에서 **vaspServer.resetNonce()** 필수: evm_revert 후 체인 nonce는 롤백되지만 NonceManager 내부 카운터는 그대로. 재동기화하지 않으면 "nonce too high" 에러 발생.

[6/7] 정적 데이터 시드

테스트가 공유할 유저·정책 데이터를 beforeAll에서 한 번만 INSERT

```
// 사용자 5명 - user-mock-001 ~ user-mock-005 (burst 시나리오 대비)
for (let i = 1; i <= 5; i++) {
  await pool.query(
    `INSERT INTO user_wallet_mapping (user_id, wallet_addr, vasp_type, verified)
    VALUES ($1, $2, 'ANVIL', true) ON CONFLICT (user_id) DO NOTHING`,
    [`user-mock-${String(i).padStart(3, '0')}`, OPERATOR_ADDR],
  );
}

// 발행 정책 - WALK_GOAL_MET 이벤트 발생 시 tokenId=1001 NFT 1개 발행
await pool.query(
  `INSERT INTO issuance_policies (event_type, token_id, amount) VALUES ($1, $2, 1)`,
  [TEST_EVENT_TYPE, TOKEN_ID],
);
```

beforeAll vs beforeEach: 변하지 않는 기준 데이터(유저, 정책)는 beforeAll에서 한 번만. TX 결과(issuance_requests, user_nft_holdings 등)는 beforeEach에서 TRUNCATE.

PgHybridCoreBankingAdapter

테스트 전용 클래스. `getUserAccount()` 만 로컬 DB(`user_wallet_mapping`)에서 조회. `recordNftHolding()` / `recordAuditLog()` 는 Java 컨테이너 경유 — 운영과 동일 경로.

[7/7] 서비스 조립 ① — TokenIssuerFactory

의존성 순서를 캡슐화한 Factory가 전체 발행 파이프라인을 조립

```
const factory = new TokenIssuerFactory({ chainAdapter, vaspAdapter: externalVasp, coreBanking, pool });
const { issuerService, confirmHandler, txStateMachine } = factory.createNFTIssuer(mockVaspAddr, conditionSvc);
```

Factory가 내부에서 만드는 것

- PgTxRepository · PgIssuanceRequestRepository
- LedgerService · IssuancePolicyService
- TxStateMachineService (EventEmitter 상속)
- TxTransitionBridge → .attach() 이벤트 구독
- IssuerService (의존성 7개)
- IssuanceConfirmHandler

왜 Factory를 쓰나?

의존성 순서가 복잡하다. TxTransitionBridge는 TxStateMachineService보다 뒤에 만들어야 하고, attach()는 둘 다 존재한 뒤에 호출해야 한다.

Factory가 이 순서를 캡슐화. 호출자는 결과물만 받는다.

TxStateMachineService는 **EventEmitter**를 상속한다. 상태가 전이될 때마다 `emit('transition', { from, to })` 를 발생시키고, TxTransitionBridge가 이를 구독해서 세 테이블(tx_mint_requests · mint_requests · issuance_requests)을 cascade 업데이트한다.

[7/7] 서비스 조립 ② — ChainEventListener

온체인 Issued 이벤트 풀링 → 두 핸들러 순서대로 호출

```
chainListener = new ChainEventListener(  
    chainAdapter,  
    [confirmHandler, new ProcessedEventHandler(mockVaspAddr, ledgerService)], // 핸들러 2개  
    [{ addr: mockVaspAddr, abi: MOCK_VASP_ABI, eventNames: ['Issued'] }],  
    { async getLastProcessedBlock() { return inMemoryBlockStore.block; }, ... },  
);  
await chainListener.start();
```

IssuanceConfirmHandler

- txHash로 tx_mint_request 조회
- handleMined() → handleConfirmed() 호출
- emit('transition', CONFIRMED) 발생
- Bridge가 issuance_requests CONFIRMED 업데이트
- audit_log(ISSUANCE_CONFIRMED) 자동 기록

ProcessedEventHandler

- 같은 Issued 이벤트를 두 번째로 수신
- (txHash, logIndex) 쌍을 processed_events에 기록
- 중복 이벤트 처리 방지 마킹
- 이미 처리된 이벤트 재수신 시 무시

Issued 이벤트 하나에 두 핸들러가 순서대로 반응한다. IssuanceConfirmHandler(상태 전이) → ProcessedEventHandler(중복 방지 마킹).

[7/7] 서비스 조립 ③ — ConsumerGroupPool

Redis Stream 두 Consumer Group 독립 소비

```
consumerPool = new ConsumerGroupPool(  
    redisAdapter, streamDlq,  
    { streamKey: 'kyobo:events', batchSize: 10, blockMs: 500, minIdleMs: 1_000 },  
    [  
        { groupName: 'nft-consumers', consumerId: 'nft-1', processors: [nftIssuedProcessor] },  
        { groupName: 'activity-consumers', consumerId: 'activity-1', processors: [activityProcessor] },  
    ],  
);  
consumerPool.start(); // 논블로킹 - 백그라운드 풀링 루프 시작
```

activity-consumers

- 교보 앱 ACTIVITY_ACHIEVED 이벤트 소비
- ActivityProcessor → IssuerService.issueActivityNFT()
- 정책 조회 → AML → TX 제출 → SUBMITTED

nft-consumers

- VASPServer NFT_ISSUED 콜백 이벤트 소비
- NFTIssuedProcessor → PgNFTLedgerService.creditNFT()
- user_nft_holdings INSERT (Java 경유)

minIdleMs = 1,000ms: XAUTOCLAIM 대상 PEL 항목 기준. 1초 이상 XACK 없이 방치된 메시지를 다른 Consumer가 재수신할 수 있다. Sepolia는 300,000ms (5분) — 실제 네트워크 지연 고려.

beforeAll 완료 — 전체 인프라 연결 상태

이 시점부터 11개 시나리오 테스트가 실행된다

블록체인 레이어

Hardhat 로컬 노드 :8545
MockVASP 컨트랙트 배포 완료
VASPServer :19876 (OPERATOR_KEY)
controlVasp (DEPLOYER_KEY)

데이터 레이어

PostgreSQL (schema 적용 완료)
Java internal-ledger :8080
Redis :6379 (Consumer Group 생성)
user 5명 + issuance_policy 시드

서비스 레이어

IssuerService (의존성 7개 주입)
TxTransitionBridge (이벤트 구독 중)
ChainEventListener (폴링 중)
ConsumerGroupPool (소비 중)

beforeEach에서는 issuance_requests / tx_mint_requests / mint_requests / processed_events / user_nft_holdings / audit_log / reconcile_history를 TRUNCATE 하고
vaspServer.resetNonce() 를 호출한다. 인프라는 유지, TX 결과만 초기화.

시나리오	검증 포인트	핵심 메커니즘
[1] NORMAL	CONFIRMED + processed_events + audit_log	전체 경로 E2E
[2] REVERT	issuance_requests FAILED	VASPServer 500 전파
[4] PENDING	mempool 체류 → CONFIRMED	freezeMining + mineBlock
[stream-4] XAUTOCLAIM	PEL 재수신 → ledger 반영	minIdleMs 초과 재할당
[burst]	5명 동시 전체 CONFIRMED	NonceManager 직렬화

mock-vasp vs Sepolia — beforeAll 차이점

구조는 동일 · 환경 제약에 따른 7가지 차이

항목	mock-vasp (Anvil)	Sepolia
블록체인 기동	GenericContainer Hardhat 컨테이너 직접 기동	환경변수 <code>SEPOLIA_RPC_URL</code> — 컨테이너 없음
MockVASP 주소	매 실행마다 새로 배포 → 새 주소	<code>SEPOLIA MOCK_VASP_ADDR</code> 고정 주소 재사용
TOKEN_ID	<code>'1001'</code> 상수	<code>String(Date.now())</code> — 이전 실행 잔액 충돌 방지
controlVasp 클래스	<code>AnvilVASPAdapter</code> — <code>freezeMining</code> / <code>snapshot</code> 포함	<code>SepoliaVASPAdapter</code> — 체인 제어 RPC 없음
minIdleMs	1,000ms	300,000ms (5분) — 이전 PEL의 의도치 않은 XAUTOCLAIM 방지
beforeEach drain timeout	3,000ms (짧게 포기)	120,000ms — Sepolia 블록타임 고려
체인 제어 RPC	사용 가능 <code>evm_setAutomine</code> / <code>evm_snapshot</code> / <code>evm_revert</code>	지원 안 됨 → DB 상태 직접 주입으로 대체

Sepolia에서 [4] **PENDING**: `evm_setAutomine` 불가 → 자연 발생 mempool pending 구간(`blockNumber=null`) 포착으로 대체

Sepolia에서 [5] **REORG**: `evm_snapshot` 불가 → `UPDATE tx_mint_requests SET status='REORGED'` DB 직접 주입으로 대체